

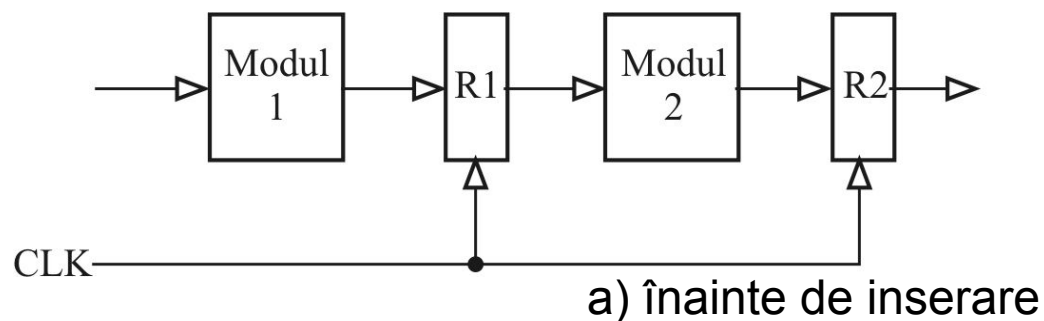
Structuri *pipeline* aritmetice

3. Controlul structurilor statice

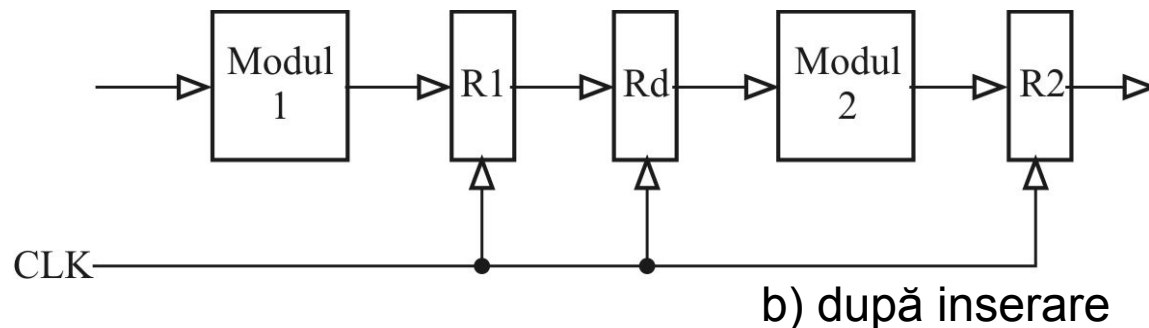
F. Reducerea LMM prin inserare de întârzieri

Uneori, prin inserare de întârzieri (registre de *delay*), este posibilă modificarea tabelului de rezervare aferentă unei structuri *pipeline* astfel încât structura în ansamblu rămâne nemodificată, dar performanțele ei cresc (**LMM** se reduce).

Modificarea structurii *pipeline* și a tabelului de rezervare aferente prin inserarea unui registru de *delay*:



Modul 1	X	
Modul 2		X



Modul 1	X		
Modul 2		d	X

Structuri *pipeline* aritmetice

3. Controlul structurilor statice

F. Reducerea LMM prin inserare de întârzieri

În ce cazuri e posibilă reducerea LMM ?

Răspuns: Notăm $n_{\max}$ numărul de semne X de pe rândul cel mai populat cu semne X în tabela de rezervare.

$$\text{LMM} \geq n_{\max}$$

Deci, $n_{\max}$ reprezintă limita minimă pe care o poate atinge latența.

Care este algoritmul care indică locul în care vor fi inserate întârzierile pentru a reduce LMM la valoarea $n=n_{\max}$?

Răspuns: Pornind de la tabela de rezervare inițială se construiește tabela revizuită.

Copiem primul semn X din tabela inițială pe aceeași poziție în tabela revizuită. Pornind de la acest prim semn X spre dreapta (pe rândul respectiv) marcăm fiecare a n -a căsuță cu I (interzis). Marcarea cu I indică rezervarea căsuțelor respective pentru următoarele inițieri; următoarele *task*-uri vor fi inițiate periodic la intervale de n perioade de tact (n cicluri *pipeline*).

Se repetă operația 1, succesiv, pentru toate semnele X din tabela inițială (baleind tabela de la stânga la dreapta) până când un semn I "cade" peste un semn X. Coliziunea se rezolvă marcând casuța de coliziune cu D –*delay* (nu cu I !) și mutarea semnului X în cauză spre dreapta, în prima căsuță liberă găsită. Aceeași deplasare se aplică (automat!) tuturor semnelor X următoare. Casuța marcată cu D în tabela revizuită devine registru de *delay* (Rd) în structura *pipeline* revizuită.

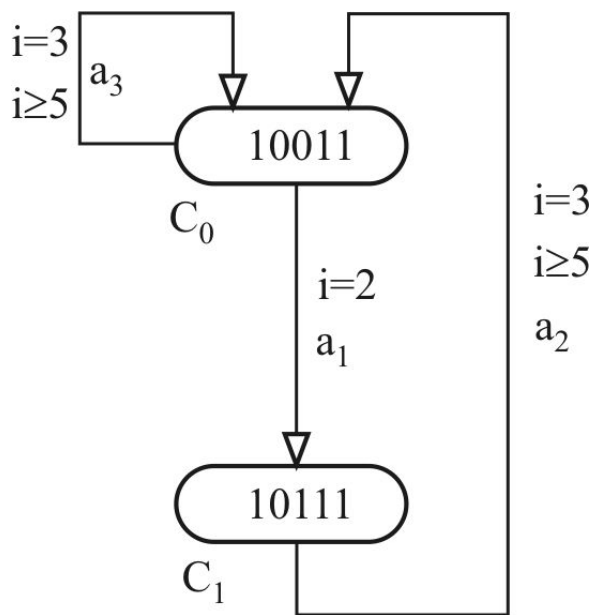
Structuri *pipeline* aritmetice

3. Controlul structurilor statice

F. Reducerea LMM prin inserare de întârzieri / Exemplu

	t_0	t_1	t_2	t_3	
Modul 1	x				x
Modul 2		x	x		
Modul 3				x	

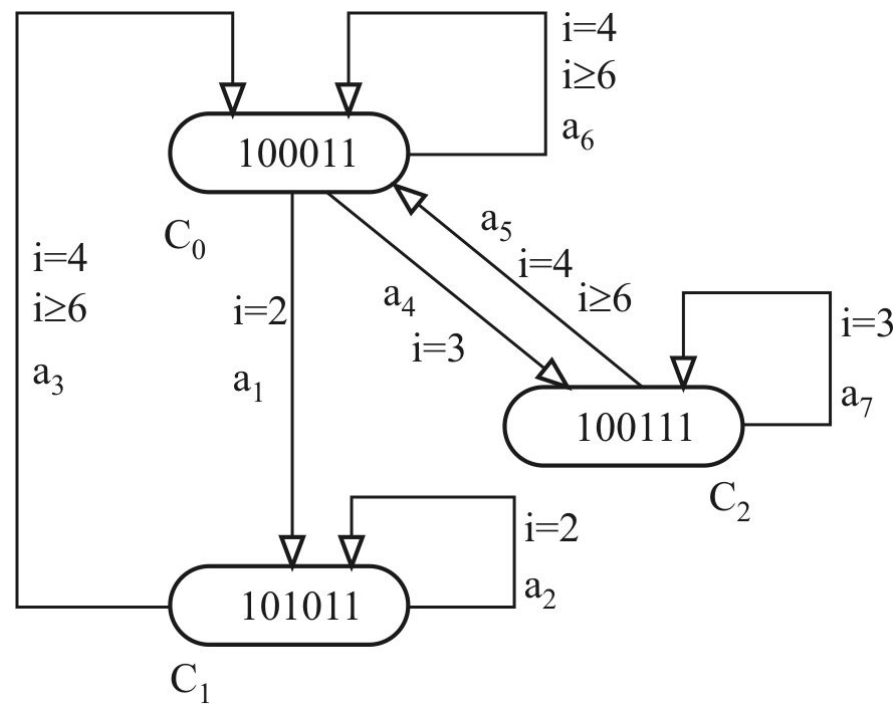
Lista latențelor interzise: $F=(4,1,0)$



Structura *pipeline* inițială
(anterior exemplificată)

	t_0	t_1	t_2	t_3	t_4	t_5
Modul 1	x				D	x
Modul 2		x	x			
Modul 3				x		

Lista latențelor interzise: $F=(5,1,0)$

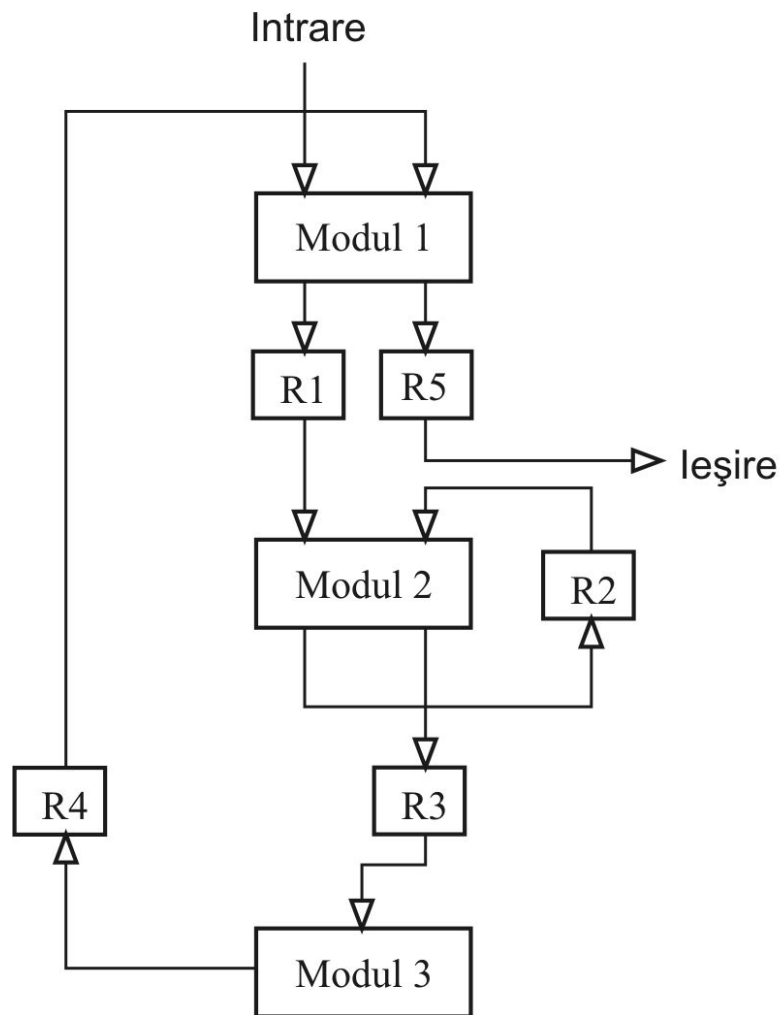


Structura revizuită prin inserare de *delay*

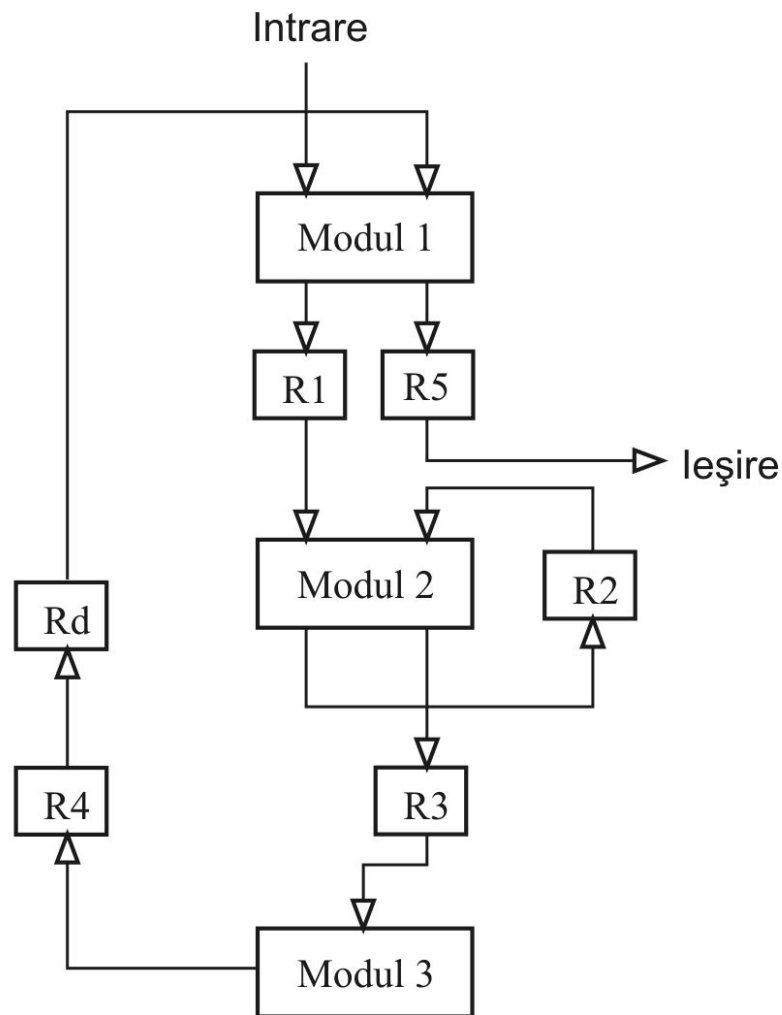
Structuri *pipeline* aritmetice

3. Controlul structurilor statice

F. Reducerea LMM prin inserare de întârzieri / Exemplu



Structura *pipeline* inițială
(anterior exemplificată)

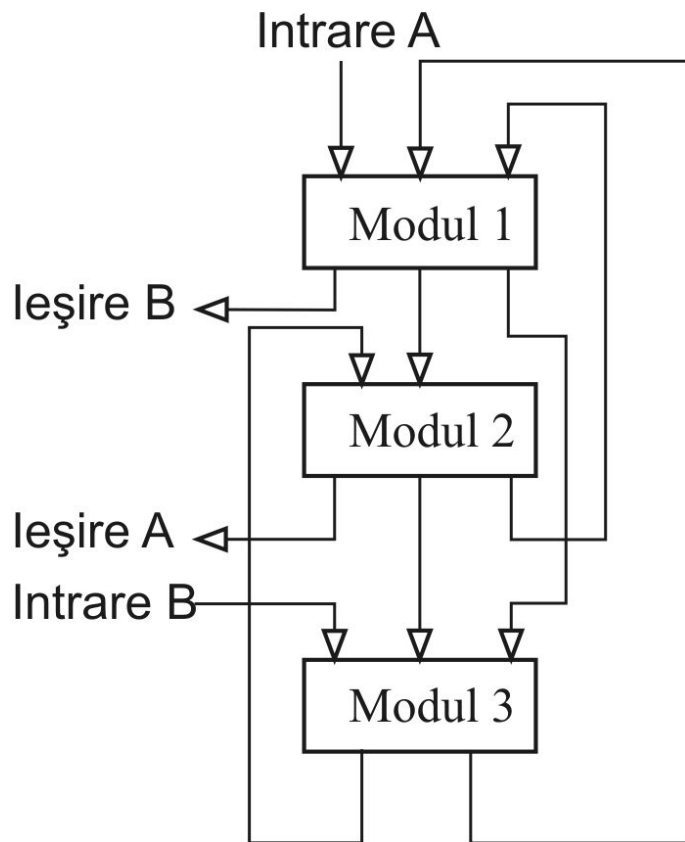


Structura rezultată prin revizuire
(inserare de *delay*)

Structuri *pipeline* aritmetice

4. Controlul structurilor dinamice

A. Tabele de rezervare



	t_0	t_1	t_2	t_3	
Modul 1	t_0 A			A	
Modul 2		A			A
Modul 3			A		

Tabela de rezervare pentru funcția A

	t_0	t_1	t_2	t_3	
Modul 1	t_4	B			B
Modul 2				B	
Modul 3	B		B		

Tabela de rezervare pentru funcția B

Structura *pipeline* dinamică partajată de două funcții (*task-uri*) A și B, cu tabelele de rezervare aferente

Structuri *pipeline* aritmetice

4. Controlul structurilor dinamice / B. Listele latențelor interzise

Pot apărea 4 succesiuni posibile la intrarea structurii *pipeline* dinamice:

AA - funcția A urmată de o altă funcție A

BA - funcția B urmată de o funcție A

AB - funcția A urmată de o funcție B

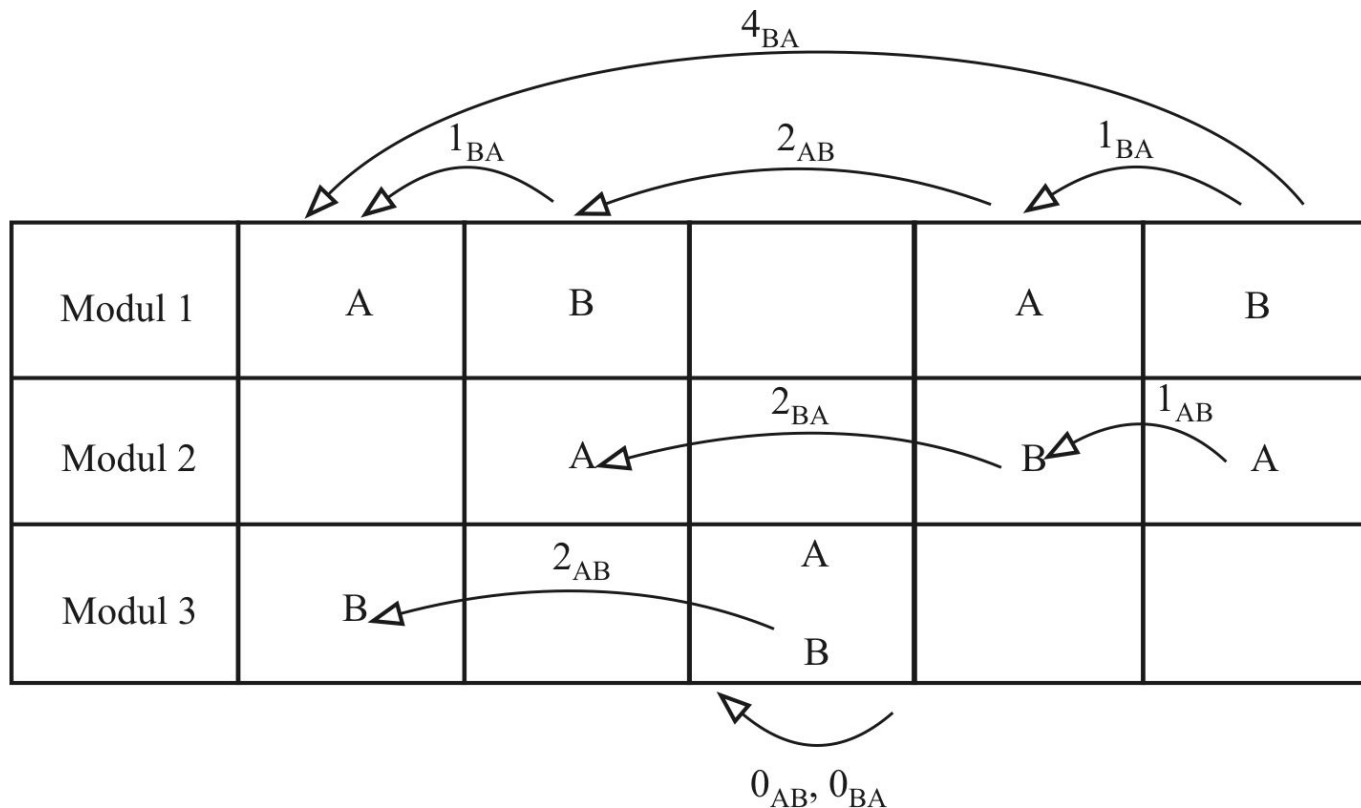
BB - funcția B urmată de o altă funcție B

Corespunzător celor 4 tipuri de succesiuni, vor exista 4 liste de latențe interzise:

$$AA = (3,0) \quad AB = (2,1,0) \quad BA = (4,2,1,0) \quad BB = (3,2,0)$$

Listele AA și BB se obțin ca în cazul structurilor statice. Listele AB și BA se obțin prin combinarea celor două tabele de rezervare (tabela A și tabela B):

Tabela combinată:



Structuri *pipeline* aritmetice

4. Controlul structurilor dinamice

C. Vectori de coliziune și matrici de coliziune

Vectorii de coliziune sunt determinați aplicând procedeul prezentat pentru structurile statice:

$$C_{AA} = (01001)$$

$$C_{BA} = (10111)$$

$$C_{AB} = (00111)$$

$$C_{BB} = (01101)$$

Vectorii de coliziune aferenți funcției A formează matricea de coliziune $\mathbf{M}_A$:

$$\mathbf{M}_A = \begin{bmatrix} C_{AA} \\ C_{AB} \end{bmatrix} = \begin{bmatrix} 01001 \\ 00111 \end{bmatrix}$$

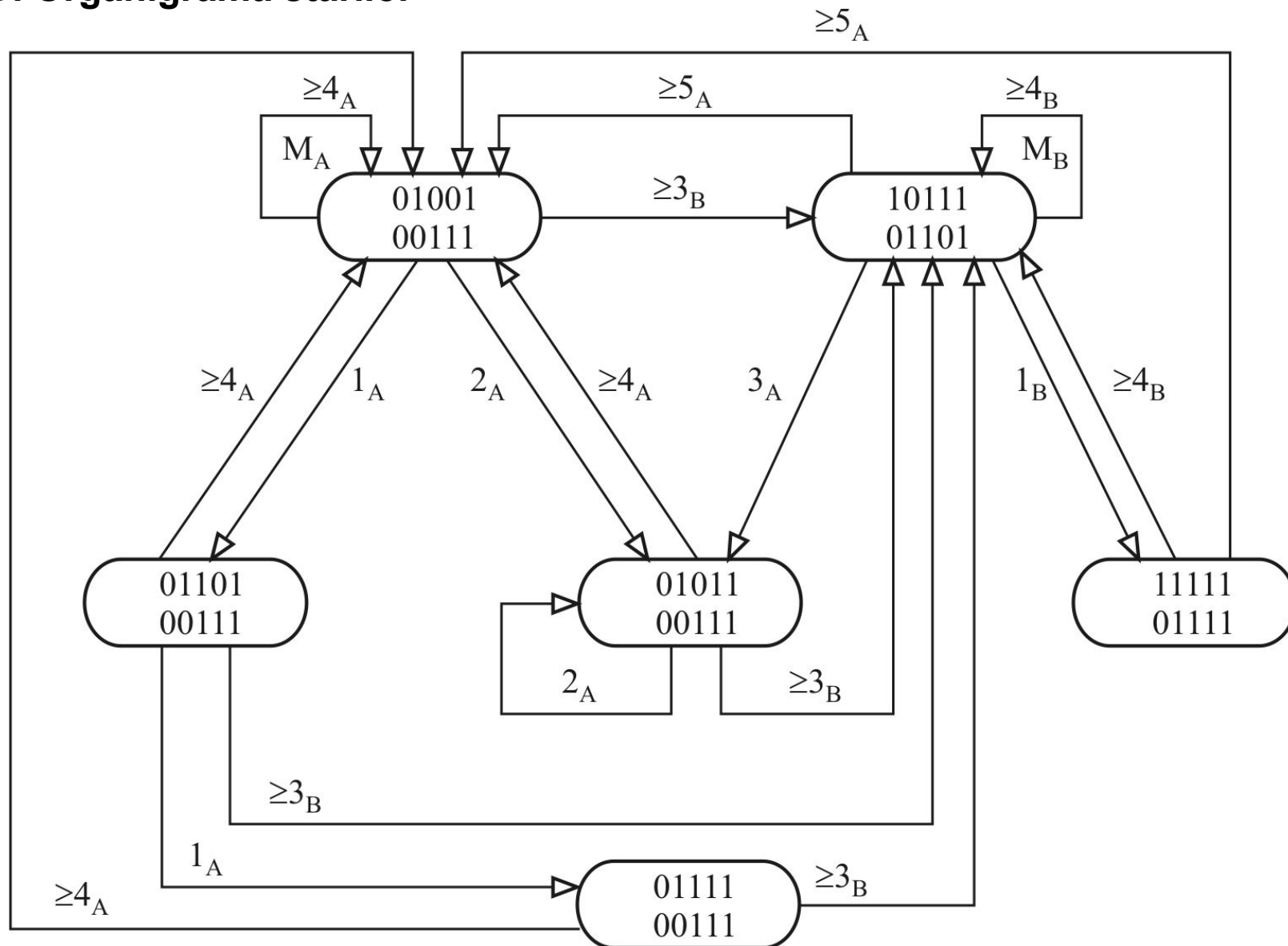
Vectorii de coliziune aferenți funcției B formează matricea de coliziune $\mathbf{M}_B$:

$$\mathbf{M}_B = \begin{bmatrix} C_{BA} \\ C_{BB} \end{bmatrix} = \begin{bmatrix} 10111 \\ 01101 \end{bmatrix}$$

Structuri *pipeline* aritmetice

4. Controlul structurilor dinamice

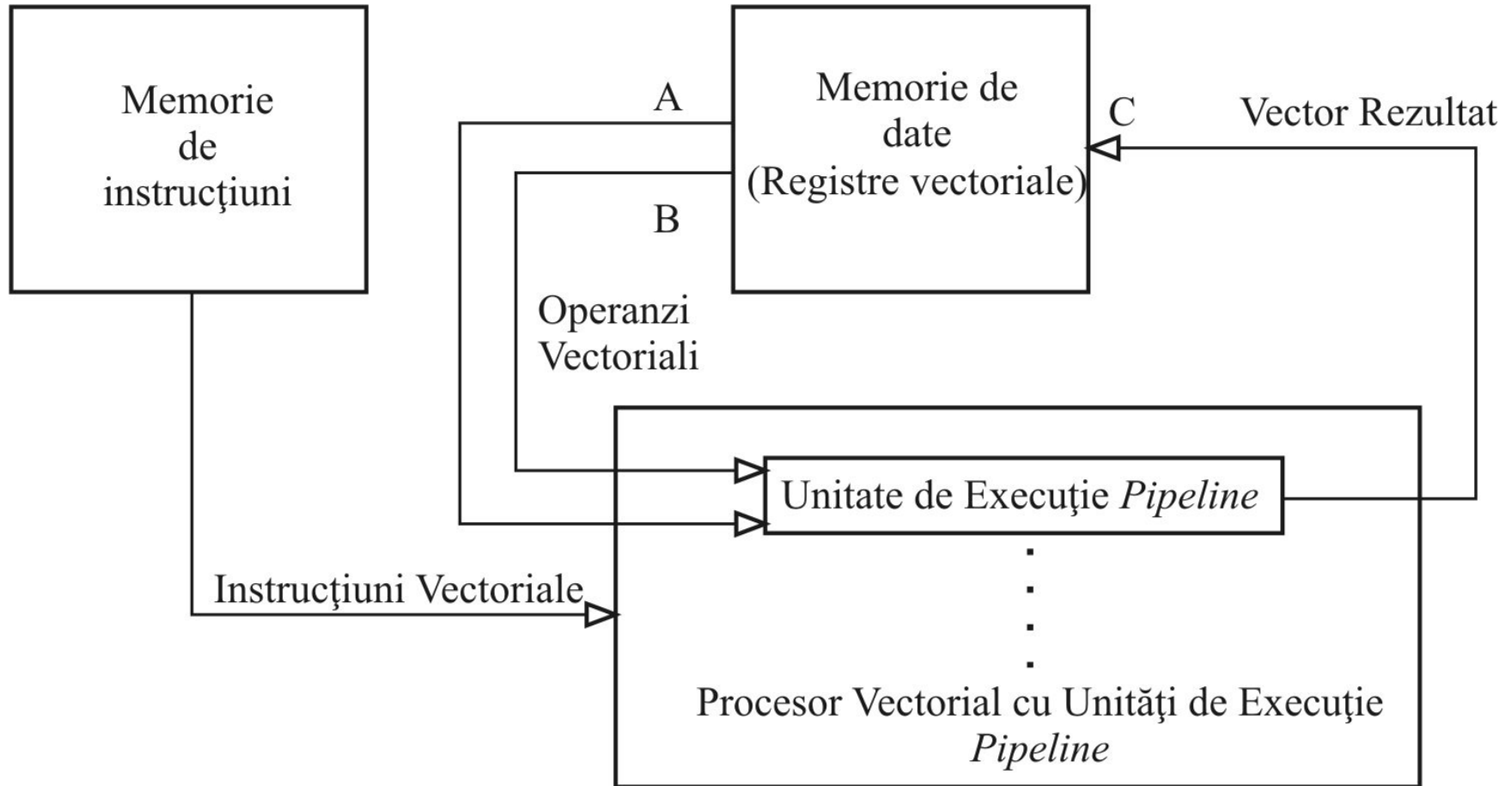
D. Organigrama stărilor



Procesarea *pipeline* în procesoarele vectoriale

1. Considerații generale

Un vector este constituit dintr-un șir de elemente scalare: $\mathbf{X} = (x_1, x_2, x_3, \dots, x_n)$



Procesarea vectorilor în tehnică *pipeline* în procesoarele vectoriale

Procesarea *pipeline* în procesoarele vectoriale

1. Considerații generale

Pentru obținerea unor performanțe superioare unitățile de execuție *pipeline*-izate (din structura procesorului vectorial) pot fi cascadeate și sincronizate.

Pentru exemplificare vom lua în considerare următoarea secvență de instrucțiuni vectoriale:

ADD RV1, RV2, RV3	; RV1 = RV2 + RV3
MUL RV5, RV4, RV1	; RV5 = RV4 * RV1

Pentru execuția celor 2 instrucțiuni procesorul vectorial va activa 2 unități de execuție:

P1 - structură *pipeline* de adunare

P2 - structură *pipeline* de înmulțire

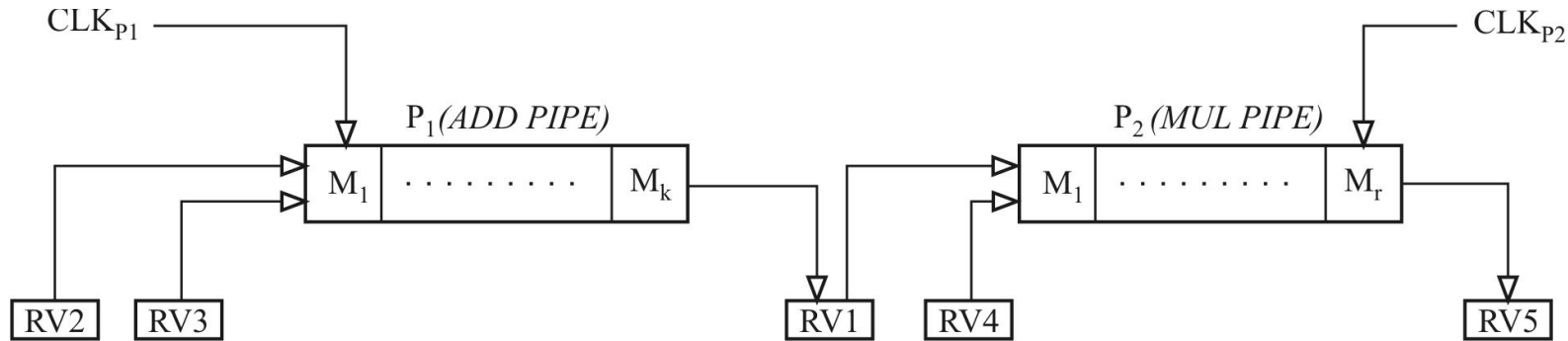
Secvența poate fi executată:

- a) fără cascadearea și sincronizarea structurilor *pipeline*
- b) cu cascadearea și sincronizarea structurilor *pipeline*

Procesarea *pipeline* în procesoarele vectoriale

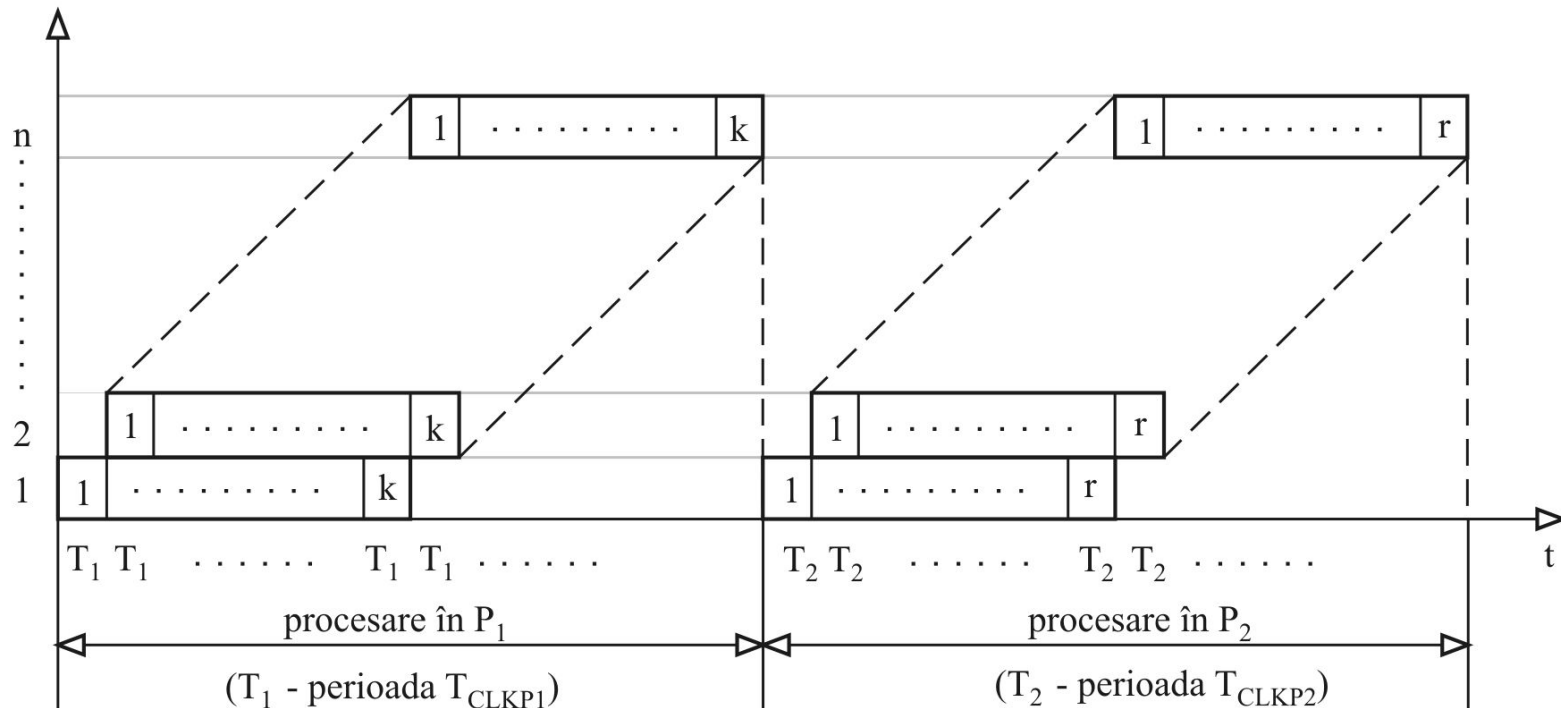
1. Considerații generale

Schema de principiu:



Componenta vectorială procesată

Diagrama spațiu-timp:

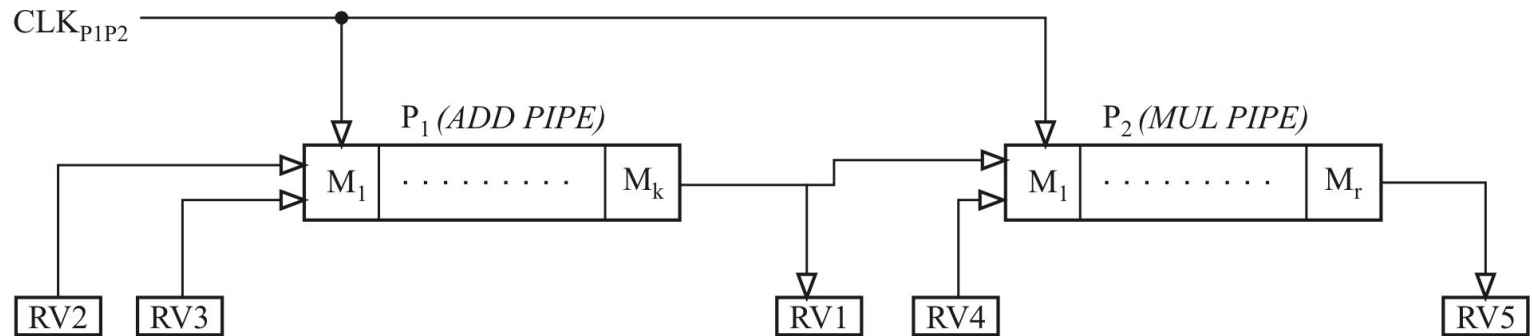


Procesarea vectorilor fără cascadatarea și sincronizarea unităților de execuție *pipeline*

Procesarea *pipeline* în procesoarele vectoriale

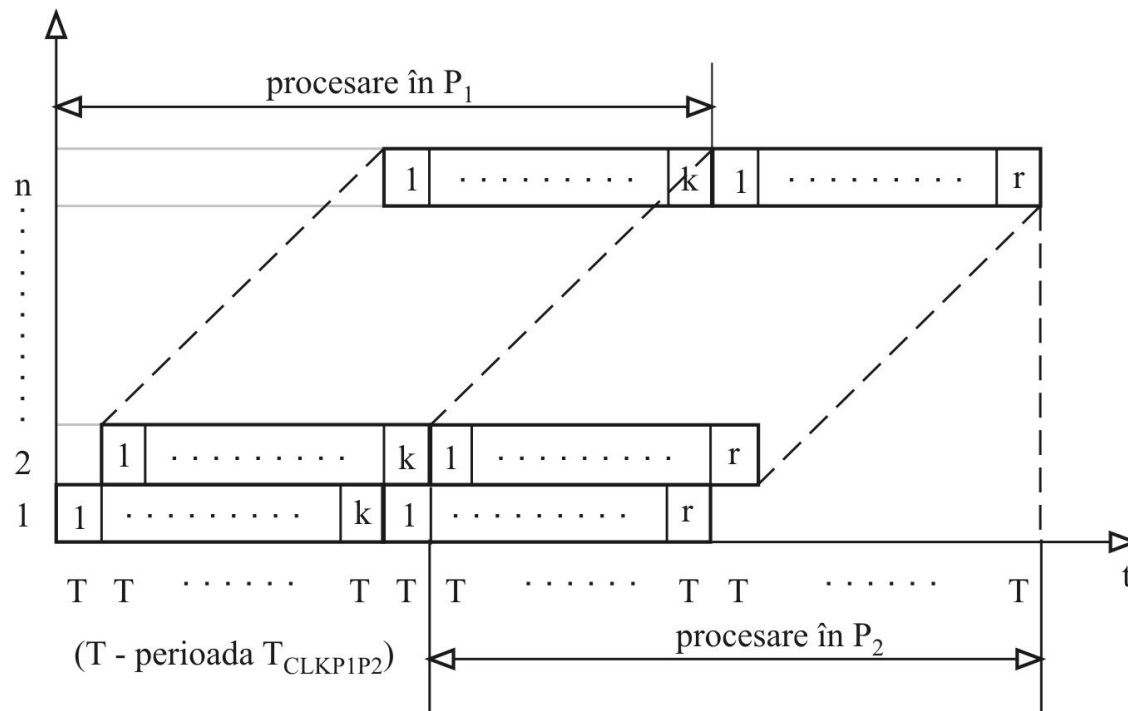
1. Considerații generale

Schema de principiu:



Componenta vectorială procesată

Diagrama spațiu-timp:



Procesarea vectorilor cu cascada și sincronizarea unităților de execuție *pipeline*